Product Requirements Document

High-Throughput Matching Engine & Market Simulator

1. The Brief

The objective is to build a deterministic, high-performance trade clearing engine paired with an agent-based synthetic market simulator. The system must process limit and market orders with strict price-time priority, maintain an in-memory Central Limit Order Book (CLOB / CBOM), and broadcast market data to downstream consumers.

The project will be implemented in either Go or Rust for the core matching engine, with an optional Python orchestration layer to run high-fidelity market simulations based on the ABIDES (Agent-Based Interactive Discrete Event Simulation) framework.

2. Core Engine Requirements

- **Order Ingestion:** Expose an HTTP or raw TCP endpoint for submitting orders (Limit, Market, Cancel).
- **Order Book Mechanics:** Implement a strict price-time priority matching engine. The data structure must optimize for O(1) or O(log N) insertions, cancellations, and matches.
- **Concurrency & State Management:** The engine must avoid traditional mutex locking on the critical hot path. The system should evaluate lock-free patterns or bounded message passing inspired by the LMAX Disruptor architecture to maximize CPU cache locality and avoid thread contention.
- **Market Data Egress:** Expose a WebSocket feed to broadcast public market data (Level 2 order-book snapshots, trade ticks, and BBO—Best Bid and Offer updates).
- **Persistence:** Asynchronously drain matched trades and order state to a durable datastore (e.g., PostgreSQL) without blocking the main execution thread.

3. Simulation Requirements (The ABIDES Layer)

- **Synthetic Agent Generation:** Build a test harness capable of spawning thousands of dummy traders (Zero-Intelligence agents, Mean-Reversion agents, and Momentum agents) to bombard the engine with realistic order flow.
- **Latency Benchmarking:** The simulation environment must timestamp orders at generation, ingestion, matching, and egress to measure the exact latency of the internal matching logic under chaotic load.

4. Performance & Load Targets

- **Throughput:** The engine must sustain a minimum of 15,000 to 20,000 orders per second (RPS) to ensure stability under heavy synthetic load.
- **Latency:** The internal matching engine target is sub-millisecond median latency, with strict control over tail latency (p99 < 5ms).
- **Stability:** The memory footprint must remain stable without catastrophic garbage collection pauses (if using Go) or memory leaks (if using Rust).

5. Deliverables

- Source code for the matching engine and WebSocket egress.
- Source code for the simulation harness/agents.
- Load test evidence (profiling data, latency histograms, and RPS metrics) demonstrating the engine's performance against the synthetic ABIDES load.

6. References

- https://blog.saikat.in/i-had-no-idea-what-an-order-book-was-now-i-ve-built-one-that-handles-18k-rps
- https://github.com/Kautenja/limit-order-book/blob/master/notes/lob.md
- https://www.youtube.com/watch?v=iwRaNYa8yTw&list=PL8nBmR5eGh34AynaXik3rgiW3qK6FKXVq
- https://www.youtube.com/watch?v=Kl4-VJ2K8Ik
- https://www.youtube.com/watch?v=C24m5WEYWxE
- https://www.youtube.com/watch?v=ph2awKa8r5Y
- https://medium.com/@farukhmahammad199/understanding-lmax-architecture-a-high-performance-event-driven-system-beb8710a40cf
- https://medium.com/@gwrx2005/the-polymarket-api-architecture-endpoints-and-use-cases-f1d88fa6c1bf
- https://github.com/solarpx/limitbook

7. Final Outcome & Future Goals

Final Outcome (System & Ops): The ultimate objective of this exercise is to achieve a highly optimized system design for a high-fidelity trading engine or Polymarket engine. To support this, the final system will feature an autonomous AI Ops layer designed to monitor all pods and automatically write basic fixes and patch vulnerabilities as needed.

Future Roadmap (1-Year Horizon):

- **Agent Collaboration:** Improve ABIDES to support mutual party formation and information-sharing groups for mutual benefit.
- **Market Manipulation Analysis:** Observe and analyze how these agent groups manipulate the markets, store the notified alarms, and study the specific techniques used.
- **Advanced Algorithmic Trading:** Improve the overall algorithmic trading capabilities of the agents in the simulator.